Resumo – Tratamento de Exceções

1. O problema

Até aqui aprendemos a ler dados com `input` e convertê-los com `int` e `float`, validar valores com `if` e organizar o código em funções e classes. Mas sempre supomos que **o usuário digita o que esperamos**.

O exemplo da aula: um programa que divide a conta dos pães de queijo (R$ 4,00 cada) entre amigos.

```
quantidade = int(input("Paes de queijo: "))
total = quantidade * 4.00
pessoas = int(input("Pessoas: "))
print(f"Cada um paga R$ {total / pessoas:.2f}")
```

```
Paes de queijo: 3
Pessoas: 2
Cada um paga R$ 6.00
```

Funciona, quando o usuário digita exatamente o que esperamos. Se ele digitar “três”:

```
Paes de queijo: tres
Traceback (most recent call last):
  File "conta.py", line 1, in <module>
    quantidade = int(input("Paes de queijo: "))
ValueError: invalid literal for int() with base 10: 'tres'
```

O programa é **interrompido**: as linhas seguintes nunca são executadas. Como evitar que o programa pare de forma inesperada?

2. Erro de sintaxe versus exceção

Erro de sintaxe	Exceção
o código **nem executa**	ocorre **durante a execução**
ex.: `if` sem dois-pontos	ex.: `int("tres")`
`SyntaxError: expected ':'`	`ValueError`, `ZeroDivisionError`, ...
corrige-se **reescrevendo o código**	trata-se com `try` / `except`

Atenção

Tratamento de exceções **não resolve erro de sintaxe**. Se o código está mal escrito, o Python recusa o programa inteiro antes de executar a primeira linha. Primeiro, o código precisa estar escrito corretamente.

Lendo a mensagem de erro

Com 3 pães de queijo e 0 pessoas, o Python mostra:

```
Traceback (most recent call last):
  File "conta.py", line 4, in <module>
    print(f"Cada um paga R$ {total / pessoas:.2f}")
ZeroDivisionError: float division by zero
```

Macete

Leia o *Traceback* **de baixo para cima**. A última linha diz o **tipo** da exceção (`ZeroDivisionError`) e **o que falhou** (divisão por zero). Logo acima está a **linha** do código onde aconteceu (`line 4`). Esse nome de tipo é exatamente o que você vai escrever no `except`.

Exercício da aula: o que acontece em cada caso?

Entradas	O que acontece
3 e 2	Cada um paga R$ 6.00
três e 2	`ValueError` na linha 1
3 e 0	`ZeroDivisionError` na linha 4
3 e duas	`ValueError` na linha 3

Nos três últimos casos, o programa é interrompido. Repare que "3 e duas" falha na linha 3, e não na 1: a primeira conversão deu certo.

3. A ideia do try/except

- `try` marca o trecho que **pode falhar**;
- `except` diz **o que fazer** se a exceção acontecer.

Nos exemplos deste resumo, as leituras foram substituídas por textos fixos para que o código possa ser executado e a saída conferida. Na prova, onde aparece `texto = "tres"`, imagine `texto = input(...)`.

```python
def conta(texto_qtd, texto_pessoas):
    try:
        quantidade = int(texto_qtd)
        total = quantidade * 4.00
        pessoas = int(texto_pessoas)
        print(f"Cada um paga R$ {total / pessoas:.2f}")
    except:
        print("Nao foi possivel calcular a conta.")
    print("Obrigado pela preferencia!")


conta("3", "2")
conta("tres", "2")
```

```
Cada um paga R$ 6.00
Obrigado pela preferencia!
Nao foi possivel calcular a conta.
```

```
Obrigado pela preferencia!
```

O que acontece na execução com "tres": `int("tres")` gera um `ValueError`; **o restante do `try` é pulado**; o `except` é executado; e o programa **continua** normalmente depois do bloco.

Macete

Pense no `try` como uma rede de segurança: assim que uma linha dentro dele falha, a execução **salta** direto para o `except`, sem voltar. As linhas do `try` que vinham depois da que falhou **nunca** são executadas. O objetivo não é esconder o erro, e sim tratá-lo de forma controlada.

4. Tratando exceções específicas

O `except` genérico trata qualquer exceção do mesmo jeito. "tres" e "0 pessoas" são problemas diferentes, mas o usuário recebe a mesma mensagem: o que ele deve corrigir? **Sempre que possível, trate exceções específicas.**

```python
def conta(texto_qtd, texto_pessoas):
    try:
        quantidade = int(texto_qtd)
        total = quantidade * 4.00
        pessoas = int(texto_pessoas)
        print(f"Cada um paga R$ {total / pessoas:.2f}")
    except ValueError:
        print("Digite apenas numeros inteiros.")
    except ZeroDivisionError:
        print("Nao e possivel dividir por zero.")
    print("Obrigado pela preferencia!")


conta("tres", "2")
conta("3", "0")
```

```
Digite apenas numeros inteiros.
Obrigado pela preferencia!
Nao e possivel dividir por zero.
Obrigado pela preferencia!
```

As exceções mais comuns

Exceção	Quando ocorre	Exemplo
`ValueError`	conteúdo inválido para a conversão	`int("tres")`
`ZeroDivisionError`	divisão por zero	`total / 0`
`IndexError`	posição inexistente de uma lista	`cardapio[10]`
`FileNotFoundError`	arquivo que não existe (modo `"r"`)	`open("menu.txt")`
`TypeError`	operação incompatível com o tipo	`"R$ " + 12.0`

Macete

Associe cada exceção à ação que a provoca: **converter** → `ValueError`; **dividir** → `ZeroDivisionError`; **acessar posição** → `IndexError`; **abrir arquivo para leitura** →

`FileNotFoundError`; **misturar tipos** (texto + número) → `TypeError`. Antes de escrever o `try`, olhe cada linha e pergunte qual dessas ações ela faz.

Exercício da aula: o cardápio

O cliente escolhe um produto pelo número da opção. Cada problema deve ter uma mensagem própria.

```
cardapio = ["Sanduiche", "Suco", "Pao de queijo"]


def escolher(texto):
    try:
        opcao = int(texto)
        print(f"Voce escolheu: {cardapio[opcao - 1]}")
    except ValueError:
        print("Digite o numero da opcao.")
    except IndexError:
        print("Opcao inexistente.")


escolher("2")
escolher("dois")
escolher("5")
escolher("0")
```

```
Voce escolheu: Suco
Digite o numero da opcao.
Opcao inexistente.
Voce escolheu: Pao de queijo
```

Atenção

Repare na última linha: com a opção 0, o programa acessa `cardapio[-1]`, que é o **último** item da lista. Índices negativos são válidos em Python, então **não há exceção**! O `try` só pega o que de fato gera exceção. Para recusar a opção 0, é preciso um `if` (ou um `raise`, como veremos adiante).

5. else e finally

else: quando deu certo

O `else` só executa se o `try` terminar **sem exceção**.

```
def dividir_conta(texto):
    try:
        pessoas = int(texto)
    except ValueError:
        print("Digite um numero inteiro.")
    else:
        print(f"Conta dividida entre {pessoas} pessoas.")


dividir_conta("3")
dividir_conta("tres")
```

```
Conta dividida entre 3 pessoas.
Digite um numero inteiro.
```

finally: executa de qualquer forma

O `finally` executa **tendo ou não** ocorrido exceção. É o lugar de mensagens de encerramento e de tarefas que precisam acontecer sempre.

```python
with open("cardapio.txt", "w", encoding="utf-8") as arq:
    arq.write("Sanduiche\nSuco\nPao de queijo\n")

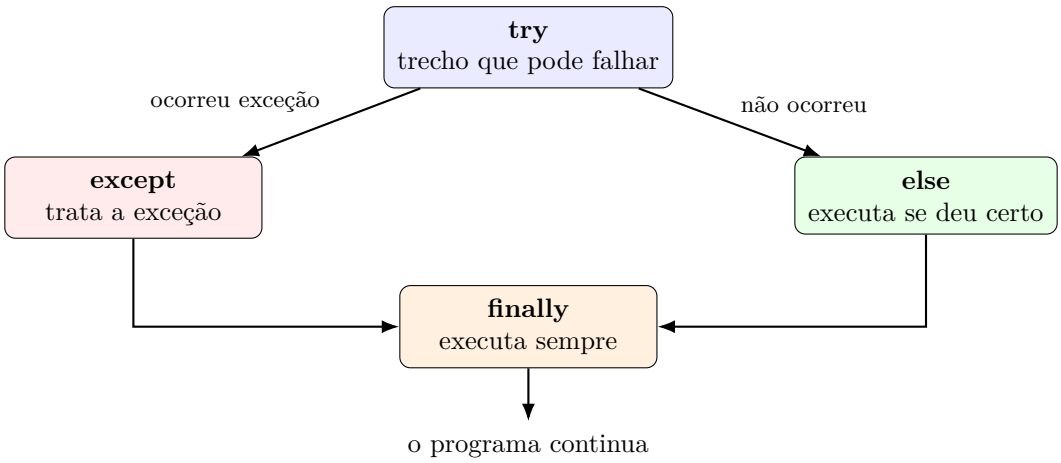

def ler_cardapio(nome):
    try:
        with open(nome, "r", encoding="utf-8") as arquivo:
            linhas = arquivo.readlines()
        print(f"Cardapio com {len(linhas)} produtos.")
    except FileNotFoundError:
        print("Cardapio nao encontrado.")
    finally:
        print("Fim da leitura do cardapio.")


ler_cardapio("cardapio.txt")
ler_cardapio("menu.txt")
```

```
Cardapio com 3 produtos.
Fim da leitura do cardapio.
Cardapio nao encontrado.
Fim da leitura do cardapio.
```

As três primeiras linhas só criam o arquivo de teste. O `with` continua fechando o arquivo automaticamente; o `readlines()` devolve uma lista com todas as linhas.

O fluxo completo



Macete

Ordem obrigatória dos blocos: `try` → `except` (um ou vários) → `else` → `finally`. Pense em **dois caminhos**: com exceção (`except`) ou sem exceção (`else`). O `finally` fica no **fim dos dois caminhos**. O mais comum é começar só com `try` e `except`.

Exercício da aula: quais blocos executam?

```python
def calcular(texto):
    print("Pessoas:", texto)
    try:
        pessoas = int(texto)
        valor = 12.00 / pessoas
    except ValueError:
        print("Entrada invalida.")
    except ZeroDivisionError:
        print("Ninguem para pagar.")
    else:
        print(f"Cada um paga R$ {valor:.2f}")
    finally:
        print("Conta encerrada.")


calcular("3")
calcular("0")
calcular("tres")
```

```
Pessoas: 3
Cada um paga R$ 4.00
Conta encerrada.
Pessoas: 0
Ninguem para pagar.
Conta encerrada.
Pessoas: tres
Entrada invalida.
Conta encerrada.
```

6. Validando a entrada com repetição

Para insistir até o usuário digitar um valor válido, combinamos `while True` com `try/except` dentro de uma função:

```python
def ler_inteiro(mensagem):
    while True:
        try:
            valor = int(input(mensagem))
            return valor
        except ValueError:
            print("Digite um numero inteiro valido.")


quantidade = ler_inteiro("Paes de queijo: ")
pessoas = ler_inteiro("Pessoas: ")
total = quantidade * 4.00
print(f"Cada um paga R$ {total / pessoas:.2f}")
```

```
Paes de queijo: tres
Digite um numero inteiro valido.
Paes de queijo: 3
Pessoas: 2
Cada um paga R$ 6.00
```

- o `while True` repete até a leitura dar certo;
- valor válido: o `return` encerra a função **e** o laço;
- valor inválido: o `except` avisa e o laço repete.

Macete

O `return` dentro do `while True` é a única saída do laço. Ele só é alcançado se a conversão na linha anterior **não** falhou. A validação fica em uma função: o programa principal fica limpo e não repete código. Mas repare: essa função aceita 0 pessoas. Para isso, existe o `raise`.

7. Lançando exceções com raise

Até agora, o Python lançava as exceções. Com `raise`, **nós** lançamos uma exceção quando um valor é inválido para as regras do problema.

```python
def dividir_conta(total, pessoas):
    if pessoas <= 0:
        raise ValueError("Numero de pessoas invalido.")
    return total / pessoas


def main(quantidade, pessoas):
    try:
        valor = dividir_conta(quantidade * 4.00, pessoas)
        print(f"Cada um paga R$ {valor:.2f}")
    except ValueError as erro:
        print(f"Erro: {erro}")


main(3, 2)
main(3, 0)
```

```
Cada um paga R$ 6.00
Erro: Numero de pessoas invalido.
```

- `raise` **lança** a exceção e **interrompe** a função: o `return` não é executado;
- `except ValueError as erro` captura a exceção e guarda na variável `erro` a mensagem escrita no `raise`;
- **a função detecta o problema; quem chamou decide como comunicá-lo.**

raise em métodos de classes

O mesmo vale para métodos. Antes, o método só imprimia um aviso, e quem chamou não ficava sabendo que a operação falhou. Com `raise`, o método avisa *quem chamou*:

```python
class Pedido:
    def __init__(self):
        self.itens = []

    def adicionar_produto(self, produto, quantidade):
        if quantidade <= 0:
            raise ValueError("A quantidade deve ser maior que zero.")
        self.itens.append([produto, quantidade])
        print(f"Adicionado: {quantidade} x {produto}")

    def exibir_resumo(self):
        print("Itens:", len(self.itens))


pedido = Pedido()
try:
    pedido.adicionar_produto("Suco", 1)
    pedido.adicionar_produto("Suco", 0)
    pedido.exibir_resumo()
except ValueError as erro:
    print(f"Erro: {erro}")
```

```
Adicionado: 1 x Suco
Erro: A quantidade deve ser maior que zero.
```

A linha `pedido.exibir_resumo()` é **pulada**, porque está dentro do `try`, depois da linha que lançou a exceção.

Macete

Diferença entre `print` e `raise` dentro de uma função: o `print` **fala com o usuário** e a função segue em frente como se nada tivesse acontecido; o `raise` **fala com o programador que chamou a função** e impede que ela continue com um dado inválido. Regra: funções e métodos **lançam**; o programa principal **trata**.

8. Boas práticas

Evite	Prefira
`except` genérico	exceções específicas
mensagens vagas	mensagens claras, que dizem o que corrigir
seguir com dados inválidos	`raise` para valores inválidos
`try` envolvendo o programa inteiro	`try` só no trecho que pode falhar

9. Erros mais comuns

Atenção

- **Achar que o try continua depois da falha:** as linhas seguintes do `try` são puladas.
- **Usar variável criada no try que falhou:** se `valor = int(texto)` falhou, `valor` não existe. Usá-la depois do bloco dá `NameError`. Use-a no `else`.

- **Errar o nome da exceção:** `except ZeroDivision:` ou `except valueerror:`. O nome precisa ser exato, com maiúsculas.
- **Esperar exceção onde não há:** `cardapio[-1]` não gera `IndexError`; abrir com `"w"` um arquivo inexistente não gera `FileNotFoundError` (ele é criado).
- **Escrever raise sem o tipo:** o correto é `raise ValueError("mensagem")`.
- **Achar que o finally é opcional na ordem:** ele vem sempre por último.

10. Questões de exemplo resolvidas

Questão 1 — Leitura de nota com intervalo (fixação 1)

Crie uma função que leia uma nota entre 0 e 100: se o usuário digitar texto, pede novamente; se estiver fora do intervalo, informa o problema e pede novamente.
Raciocínio. Junte as ideias da aula: `raise` para cada regra e repetição para insistir. Como as duas falhas viram `ValueError`, a função de validação lança cada uma com a **sua própria mensagem**, e quem chama só precisa exibir `erro`. Para poder executar aqui, as "digitações" vêm de uma lista.

```python
def validar_nota(texto):
    try:
        nota = float(texto)
    except ValueError:
        raise ValueError("Digite um numero.")
    if nota < 0 or nota > 100:
        raise ValueError("A nota deve estar entre 0 e 100.")
    return nota


def ler_nota(digitacoes):
    for texto in digitacoes:
        try:
            return validar_nota(texto)
        except ValueError as erro:
            print(erro)


print(ler_nota(["oito", "150", "85.5"]))
```

```
Digite um numero.
A nota deve estar entre 0 e 100.
85.5
```

Na versão de prova, o `for texto in digitacoes` vira `while True` com `texto = input("Nota: ")`.

Questão 2 — Cardápio em arquivo (fixação 4)

Leia o arquivo `cardapio.txt`, com uma linha por produto no formato `nome;preco`. Trate o arquivo inexistente e avise sobre as linhas com preço inválido, ignorando-as.
Raciocínio. Dois níveis de tratamento: um `try` externo para o arquivo e um `try` interno, **dentro do laço**, para cada linha. Assim, uma linha ruim não impede a leitura das demais.

```python
with open("cardapio.txt", "w", encoding="utf-8") as arq:
    arq.write("Sanduiche;15.00\nSuco;oito\nPao de queijo;4.00\n")


def carregar(nome):
    precos = {}
    try:
        with open(nome, "r", encoding="utf-8") as arquivo:
            for linha in arquivo:
                partes = linha.strip().split(";")
                try:
                    precos[partes[0]] = float(partes[1])
                except ValueError:
                    print("Preco invalido:", partes[0])
    except FileNotFoundError:
        print("Arquivo nao encontrado.")
    return precos


print(carregar("cardapio.txt"))
print(carregar("outro.txt"))
```

```
Preco invalido: Suco
{'Sanduiche': 15.0, 'Pao de queijo': 4.0}
Arquivo nao encontrado.
{}
```

Questão 3 — raise em método (fixação 3)

Altere `realizar_emprestimo` para lançar um `ValueError` quando o livro não estiver disponível. O programa principal deve tratar a exceção e exibir a mensagem.

```python
class Livro:
    def __init__(self, titulo):
        self.titulo = titulo
        self.disponivel = True

    def realizar_emprestimo(self):
        if not self.disponivel:
            raise ValueError(f"{self.titulo} nao esta disponivel.")
        self.disponivel = False
        print(f"Emprestimo de {self.titulo} realizado.")


livro = Livro("Algoritmos")
for tentativa in range(2):
    try:
        livro.realizar_emprestimo()
    except ValueError as erro:
        print(f"Erro: {erro}")
```

```
Emprestimo de Algoritmos realizado.
Erro: Algoritmos nao esta disponivel.
```

Questão 4 — Leitura de código

Escreva a saída do programa abaixo.

```
def processar(valores, i):
    try:
        print("Inicio", i)
        x = 10 // valores[i]
        print("Resultado", x)
    except ZeroDivisionError:
        print("Divisao por zero")
        return -1
    except IndexError:
        print("Posicao", i, "nao existe")
    else:
        return x
    finally:
        print("Fim", i)
    return 0


dados = [5, 0]
print(processar(dados, 0))
print(processar(dados, 1))
print(processar(dados, 2))
```

Raciocínio. Detalhe importante: o `finally` executa **mesmo quando há um** `return` no `except` ou no `else`. Ele roda antes de a função de fato devolver o valor.

- `i = 0`: $10//5 = 2$, sem exceção. O `else` devolve 2, mas antes o `finally` imprime;
- `i = 1`: $10//0$ gera `ZeroDivisionError`; o `except` devolve -1, depois do `finally`;
- `i = 2`: `valores[2]` gera `IndexError`; não há `return` nesse `except`, então o `finally` roda e a função segue até o `return 0`.

```
Inicio 0
Resultado 2
Fim 0
2
Inicio 1
Divisao por zero
Fim 1
-1
Inicio 2
Posicao 2 nao existe
Fim 2
0
```

11. Recapitulando

1. Erros de sintaxe impedem a execução; exceções acontecem durante a execução.
2. O `try/except` trata a exceção sem interromper o programa.
3. Trate exceções específicas, com mensagens claras.

4. O `else` executa quando deu certo; o `finally`, sempre.

5. O `raise` sinaliza valores inválidos em funções e métodos.

12. Cola rápida

Quero...	**Escrevo...**
proteger um trecho	`try:` ... `except TipoDaExcecao:`
tratar dois problemas	dois `except`, um para cada tipo
ler a mensagem da exceção	`except ValueError as erro:` e `print(erro)`
executar só se deu certo	`else:`
executar sempre	`finally:`
repetir até dar certo	`while True:` + `try` + `return`
lançar uma exceção	`raise ValueError("mensagem")`
texto que não é número	`ValueError`
divisão por zero	`ZeroDivisionError`
posição inexistente	`IndexError`
arquivo inexistente	`FileNotFoundError`
texto + número	`TypeError`